

## Introduction

---

Some problems look like a data structures exercise until you actually try to run your solution at scale — then they turn into a math problem in disguise. LeetCode 3116, *Kth Smallest Amount With Single Denomination Combination*, is exactly that kind of problem, and it's a favorite in hard-tier interview rounds because it tests something more valuable than syntax: can you recognize when brute force is structurally doomed, and can you reframe a problem you can't solve directly into one you can?

The setup sounds simple. You have a coin machine that only takes one denomination per payment — no mixing 3s and 4s in the same transaction. Across every denomination, sorted from smallest to largest, you need the k-th amount that could ever be paid. It's a clean question with a nasty catch: k can be up to two billion, which immediately rules out any approach that tries to construct the answer directly.

Interviewers like this problem because it forces a mental shift that shows up constantly in real systems — from rate limiters to search ranking to inventory queries. When you can't enumerate the answer space, you count instead, and you binary search on the count. Once you've internalized that pattern, a whole category of "find the k-th X" problems stops being intimidating.

## Problem Overview

---

You're given an array of coin values, and you can use unlimited coins of any single denomination in one payment — but you can never combine two different denominations. So if 3 is one of your coins, valid amounts are 3, 6, 9, 12, and so on. If 4 is also a coin, 4, 8, 12, 16 join the list too.

Now take every amount payable by *any* denomination, merge them all into one sorted list with duplicates removed, and return the k-th smallest value in that merged list.

Two constraints define the shape of the solution:

- The coin array has at most 15 elements, which matters because  $2^{15}$  is only about 32,000 — small enough to iterate over every subset of coins.
- k can be as large as 2 billion, which rules out building any list of amounts, since that list would need to hold billions of entries before you could even sort it.

## Example

---

Take coins `[3, 6, 9]` with `k = 3`.

- Multiples of 3: 3, 6, 9, 12...
- Multiples of 6: 6, 12, 18...
- Multiples of 9: 9, 18, 27...

Merge and dedupe: 3, 6, 9, 12, 15, 18... Notice that 6 and 9 don't add anything new before 12 — they're both multiples of 3, so they were already going to appear as multiples of 3 anyway. The third smallest amount in the merged list is 9.

That overlap — 6 and 9 already showing up as multiples of 3 — is the whole crux of the problem. The more coins you add, the messier this overlap gets, and untangling it by brute-force merging is where the naive approach falls apart.

## Intuition

---

The instinct most people reach for first is: generate multiples of every coin, throw them in a set to kill duplicates, sort, and index into position  $k$ . It works on the example. It will not work on the actual constraints, because  $k$  can be 2 billion — you'd be building and sorting a list with billions of entries before you could even answer the question.

The way out is to stop thinking about *generating* amounts and start thinking about *counting* them. Instead of asking "what is the  $k$ -th amount?", ask a different, answerable question: "how many valid amounts are less than or equal to some number  $x$ ?" If you can answer that count quickly for any  $x$ , you can binary search for the smallest  $x$  where the count reaches  $k$  — and that  $x$  is your answer.

This is the same shift that shows up in "find the  $k$ -th smallest element in a sorted matrix" or "find the smallest capacity that lets you ship packages within  $D$  days" — whenever direct enumeration is too expensive but a counting function is cheap and monotonic, binary search on the answer becomes the tool.

The remaining puzzle is how to count correctly. If you naively count multiples of 3 up to  $x$ , then add multiples of 4 up to  $x$ , you double-count anything that's a multiple of both — like 12. This is the classic overlap problem: 5 people won prize A, 4 people won prize B, but 2 people won both, so the real total of distinct winners is  $5 + 4 - 2$ , not 9. Multiples of 3 and 4 overlap exactly on multiples of their least common multiple (LCM), which is 12. Subtract those once, and the count is exact.

With more than two coins, this generalizes into inclusion-exclusion: add the counts for each single coin, subtract the counts for each pair (using their LCM), add back the counts for each triple, and so on, alternating sign by subset size. Every subset's LCM tells you which multiples were over- or under-counted, and the alternating signs cancel out every duplicate exactly once.

## Brute Force Solution

---

**Idea:** Generate multiples of every coin up to some bound, merge them into a set to remove duplicates, sort, and return the k-th element.

**Algorithm:** 1. Pick an upper bound large enough to guarantee at least k valid amounts exist below it. 2. For each coin, generate every multiple up to that bound. 3. Insert all multiples into a set (this naturally deduplicates overlapping multiples like 12). 4. Sort the set and index into position  $k - 1$ .

**Advantages:** Trivial to reason about, easy to verify against small examples, no math beyond basic multiplication.

**Disadvantages:** The list size scales with k itself, and k can reach 2 billion — the set and the sort both become infeasible in both time and memory.

**Complexity:** Time is roughly  $O(k \log k)$  for building and sorting, and space is  $O(k)$  for storing the amounts. Both are unworkable at the given constraints.

## Optimal Solution

---

The optimal solution has two parts working together: a **count function** that answers "how many valid amounts are  $\leq x$ " using inclusion-exclusion, and a **binary search** that uses that count function to home in on the exact k-th value.

### Count function (inclusion-exclusion over subsets of coins):

For every non-empty subset of coins, compute the LCM of that subset. The number of multiples of that LCM up to x is  $x // \text{lcm}$ . Add this count if the subset size is odd, subtract it if the subset size is even. Summing over all subsets gives the exact count of amounts  $\leq x$  that are payable by at least one coin, with no double-counting.

### Subset LCM Sign Contribution

$$\{3\} \quad 3 \quad + \quad x // 3$$

$$\{4\} \quad 4 \quad + \quad x // 4$$

$$\{3, 4\} \quad 12 \quad - \quad x // 12$$

### Binary search on x:

1. Set  $\text{lo} = 1$  and  $\text{hi}$  to a value large enough to guarantee the count reaches k (the smallest coin times k is a safe upper bound).
2. While  $\text{lo} < \text{hi}$ , compute  $\text{mid} = (\text{lo} + \text{hi}) // 2$ .

3. If `count(mid) >= k`, the answer is at or below `mid`, so set `hi = mid`.
4. Otherwise set `lo = mid + 1`.
5. When `lo == hi`, that value is the k-th smallest valid amount.

This works because the count function is **monotonically non-decreasing** as `x` grows — larger `x` can only include more valid amounts, never fewer. That monotonicity is exactly what makes binary search valid here.

## Python Code

```
from itertools import combinations
from math import lcm
from typing import List

def findKthSmallest(coins: List[int], k: int) -> int:
    n = len(coins)

    def count_at_most(x: int) -> int:
        total = 0
        for size in range(1, n + 1):
            sign = 1 if size % 2 == 1 else -1
            for combo in combinations(coins, size):
                total += sign * (x // lcm(*combo))
        return total

    lo, hi = 1, min(coins) * k
    while lo < hi:
        mid = (lo + hi) // 2
        if count_at_most(mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

## Code Walkthrough

- `count_at_most(x)` implements inclusion-exclusion: it loops over every subset size from 1 to `n`, and within each size, over every combination of coins of that size.
- `sign` alternates based on subset size — positive for odd sizes (single coins, triples...), negative for even sizes (pairs, quadruples...), which is exactly what cancels out double-counted multiples.
- `lcm(*combo)` computes the least common multiple of the coins in that subset; Python's `math.lcm` accepts multiple arguments directly, so no manual GCD chaining is needed.

- `x // lcm(*combo)` counts how many multiples of that LCM are  $\leq x$  — the raw count for that subset's contribution.
- The binary search bounds `lo = 1` and `hi = min(coins) * k` guarantee the answer is inside the range, since the k-th multiple of the smallest coin is always an upper bound on the true answer.
- The loop narrows `[lo, hi]` until they meet, using the standard "search for smallest x where predicate is true" binary search template.

## Dry Run

---

Coins `[3, 4]`, `k = 5`.

- `lo = 1`, `hi = 3 * 5 = 15`.
- `mid = 8`: `count = (8//3) + (8//4) - (8//12) = 2 + 2 - 0 = 4`. That's less than 5, so `lo = 9`.
- `mid = 12`: `count = (12//3) + (12//4) - (12//12) = 4 + 3 - 1 = 6`. That's  $\geq 5$ , so `hi = 12`.
- `mid = 10`: `count = (10//3) + (10//4) - (10//12) = 3 + 2 - 0 = 5`. That's  $\geq 5$ , so `hi = 10`.
- `mid = 9`: `count = (9//3) + (9//4) - (9//12) = 3 + 2 - 0 = 5`. That's  $\geq 5$ , so `hi = 9`.
- `lo == hi == 9`. The answer is 9.

Checking by hand: multiples of 3 are 3, 6, 9, 12...; multiples of 4 are 4, 8, 12...; merged and sorted: 3, 4, 6, 8, 9 — the 5th value is 9. Matches.

## Complexity Analysis

---

**Time:** For each binary search step, `count_at_most` iterates over all  $2^n - 1$  non-empty subsets of coins, computing an LCM per subset. With  $n$  up to 15, that's roughly 32,000 subsets per check. Binary search runs in about  $O(\log(\text{range}))$  steps — around 40 iterations for a range up to `min(coins) * k` with  $k$  up to 2 billion. Total time is  $O(2^n \cdot \log(\text{range}))$ , comfortably fast because  $n$  is capped small.

**Space:** Only  $O(n)$  for tracking the current subset during combination generation — no list of amounts is ever built.

**Why it's correct:** Inclusion-exclusion guarantees an exact count with no double-counting, and the count function is monotonic in  $x$ , which is the exact precondition binary search requires to converge on the correct boundary.

## Alternative Solutions

---

A heap-based k-way merge (repeatedly popping the smallest unseen multiple across all coins, skipping duplicates) is a middle ground — it avoids generating a fully sorted list upfront, but it still does  $O(k)$  pop operations, which is a dealbreaker when  $k$  reaches 2 billion. It's worth mentioning in an interview as a "better than brute force but still not good enough" step, since it shows you understand the scaling problem before landing on the binary-search-on-count approach.

## Edge Cases

---

- **Single coin:** With only one denomination, the count function reduces to `x // coin`, and the answer is simply `coin * k`.
- **k = 1:** The answer is always the smallest coin value, since it's the smallest amount payable by definition.
- **Coins sharing a common factor:** If every coin shares a factor (like 6, 10, 14 all being even), the LCMs and counts shift, but the binary search bounds stay valid — the smallest coin times  $k$  is still a safe upper bound.
- **Maximum coins (15) and maximum k ( $2 \times 10^9$ ):** This is the stress case that breaks brute force; the optimal solution stays fast because its cost depends on  $2^n$ , not  $k$ .
- **Duplicate LCMs across different subsets:** Since coins are guaranteed distinct, this doesn't cause incorrect cancellation, but it's worth double-checking your subset generation doesn't accidentally skip or repeat a combination.

## Common Mistakes

---

1. Forgetting to alternate the sign by subset size, which causes systematic over- or under-counting.
2. Computing LCM incorrectly by summing or averaging instead of using the proper `lcm(a, b) = a * b // gcd(a, b)` relationship (or `math.lcm` directly).
3. Setting the binary search upper bound too low, so the true answer falls outside the search range.
4. Using `count(mid) > k` instead of `count(mid) >= k` in the binary search condition, which breaks the "smallest  $x$  satisfying the predicate" invariant.
5. Attempting to actually build and sort the merged list of amounts, which silently works on small examples but times out or exhausts memory on the real constraints.

## Interview Questions

---

- What property must `count_at_most(x)` have for binary search to be valid here, and why?
- How does the time complexity change if the number of coins grows from 15 to, say, 30?
- Could you solve this with a min-heap instead of inclusion-exclusion? What's the tradeoff?

- If all coins are guaranteed to share a common factor, does that change your binary search bounds or just your counting logic?
- How would you extend this to answer "how many valid amounts exist in a range [a, b]" instead of just up to x?

## Similar Problems

---

- **LeetCode 1201 — Ugly Number III:** Nearly identical structure — count multiples of up to three numbers using inclusion-exclusion, then binary search for the k-th ugly number.
- **LeetCode 1283 — Find the Smallest Divisor Given a Threshold:** Same "binary search on the answer with a monotonic check function" pattern, applied to a division-sum constraint instead of counting.
- **LeetCode 719 — Find K-th Smallest Pair Distance:** Another k-th-value problem where direct enumeration is too slow, solved by binary searching a distance and counting pairs within it.
- **LeetCode 378 — Kth Smallest Element in a Sorted Matrix:** The same counting-instead-of-generating idea, applied to a 2D sorted grid.

## Key Takeaways

---

When the answer space is too large to enumerate but you can cheaply count how many candidates satisfy "at most x," binary search on x is almost always the move. Inclusion-exclusion is the tool for counting union sizes without double-counting overlaps, and it pairs naturally with problems built on multiples, divisors, or LCMs. The subset count ( $2^n$ ) being small here is what makes the whole approach tractable — always check which variable in the problem is actually small before deciding your algorithm's complexity around it.

## Watch the Video

---

For the full walkthrough — including a live timing comparison between brute force and the optimal solution on a real 15-coin input — watch the video here: {LINK}

## About the Series

---

This breakdown is part of *Fun with Learning Technology*, a series dedicated to building real algorithmic intuition instead of memorized solutions — tracing how an experienced engineer actually arrives at an optimal approach, one problem at a time.

## Call To Action

---

If this helped the inclusion-exclusion and binary-search-on-answer pattern click, give the video a like on YouTube and subscribe for more deep dives like this one. For the full write-ups delivered straight to your inbox, subscribe to the Substack. Drop a comment with your own approach or a problem you'd like covered next, and share this with anyone prepping for interviews.

---

## Quick Review

---

### **LeetCode 3116 — what pattern does 'kth smallest sum of subset sums' trigger?**

Binary search on the answer value + counting via inclusion-exclusion, not enumeration.

### **Time complexity of the binary search + count approach here?**

$O(2^m * \log(k * \text{maxCoin}))$  where  $m$  = number of coin denominations (small, since  $2^m$  subsets).

### **Space complexity for this solution?**

$O(m)$  for the subset enumeration/recursion stack; no large auxiliary storage needed.

### **Trickiest bug when counting payable amounts $\leq \text{mid}$ ?**

Double-counting overlaps between coin multiples — must use inclusion-exclusion over LCMs of coin subsets, with sign flips.

### **This problem vs plain 'kth smallest element in sorted array' pattern?**

Can't merge/heap because the set is implicit and huge; must flip to counting how many values  $\leq x$  exist.

### **Interview follow-up: why binary search on the answer instead of generating amounts directly?**

Amount space is exponential/unbounded; counting 'how many  $\leq x$ ' is cheap and monotonic, enabling search instead of enumeration.